

MASTER 1 · IEF · FE · ANALYSE DE DONNÉES

L'apprentissage supervisé

Principe, intuition et méthodes d'ensemble : de l'arbre de décision à la random forest

1 • PRINCIPE

Supervisé ou non : y a-t-il une cible ?

	Supervisé	Non supervisé
Cible y	Observée dans les données	Absente
Objectif	Prédire y pour de nouveaux cas	Découvrir une structure, des groupes
Exemples	Scoring de crédit, fraude, prix immobilier, départ d'un client	Segmentation de clientèle, typologie de régions
Évaluation	Directe, sur un échantillon de test	Indirecte : inertie, silhouette
Algorithmes	Logistique, kNN, arbres, forêts, boosting	k-means, CAH, ACP

1 • PRINCIPE

Deux familles selon la nature de la cible y

Classification

y est une **catégorie** : défaut ou non, fraude ou non, faillite, résiliation d'un contrat.

Le modèle donne une classe et, surtout, une **probabilité**.

Mesures : accuracy, précision, rappel, AUC

Régression

y est un **nombre** : prix d'un logement, salaire, chiffre d'affaires, volatilité.

Le modèle donne une valeur numérique.

Mesures : MAE, RMSE, R^2 sur le test

Arbres et forêts traitent les deux cas. **Aujourd'hui, et dans le TP : la régression.** La classification est en annexe.

Une banque doit estimer la valeur d'un logement



Ce qu'on sait du quartier

Revenu des habitants, âge et taille des logements, localisation, distance à l'océan...



Ce qu'on veut savoir

Combien valent ses logements ? Un **nombre**, en dollars.



Ce dont on dispose

20 640 quartiers de Californie **dont on connaît la valeur** (recensement de 1990). C'est le jeu de données du TP.

Le jeu de données : une ligne par quartier

X : les variables explicatives

y : la cible

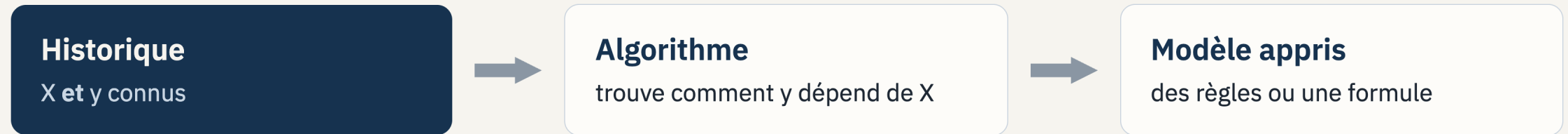
Quartier	Revenu médian	Âge médian	Pièces / ménage	Zone	Valeur médiane
1	83 k\$	41 ans	7,0	Près de la baie	452 600 \$
2	21 k\$	42 ans	4,3	Près de la baie	226 700 \$
3	68 k\$	33 ans	7,2	À moins d'1 h de l'océan	389 700 \$
4	37 k\$	33 ans	4,8	Près de l'océan	140 700 \$
...	...	...	...	...	...
Nouveau	29 k\$	30 ans	5,7	Près de la baie	?

Pour l'historique, on connaît **X et y** : c'est là-dessus qu'on apprend. Pour un nouveau quartier, on ne connaît que **X** : le modèle doit estimer le **?** Comme y est un nombre, c'est une **régression**.

1 · PRINCIPE

Apprendre par l'exemple, puis prédire

① ENTRAÎNEMENT



② PRÉDICTION



X : l'ensemble des variables explicatives (revenu médian, âge des logements, localisation...)
y : la cible à prédire (valeur médiane des logements), connue seulement pour l'historique

La régression linéaire, un modèle supervisé connu

$$y_i = \beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip} + \varepsilon_i$$

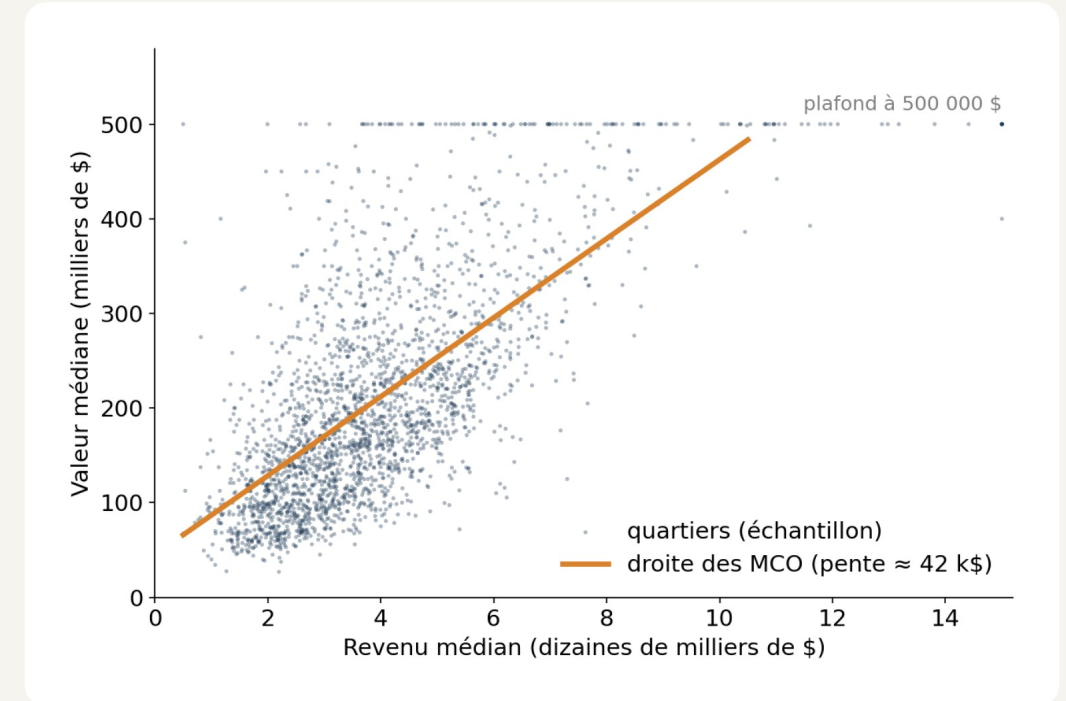
$$\hat{\beta} = \arg \min_{\beta} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_{i1} - \dots - \beta_p x_{ip})^2$$

Les MCO

minimisent la somme des carrés des résidus. Chaque coefficient : effet marginal de sa variable, toutes choses égales par ailleurs.

Déjà de l'apprentissage supervisé

y est observé, on minimise une perte quadratique, on peut prédire $\hat{y}$ pour un nouvel individu.



1 · PRINCIPE

Économétrie et machine learning : deux questions

ÉCONOMÉTRIE

« Quel est l'effet de x sur y ? »

On s'intéresse aux **coefficients estimés** : sens, taille, significativité, causalité.

APPRENTISSAGE SUPERVISÉ

« Quelle valeur pour ce nouveau quartier ? »

On s'intéresse à la **prédiction** : sa qualité sur des données jamais vues.

Une variable très prédictive n'est pas forcément une cause.

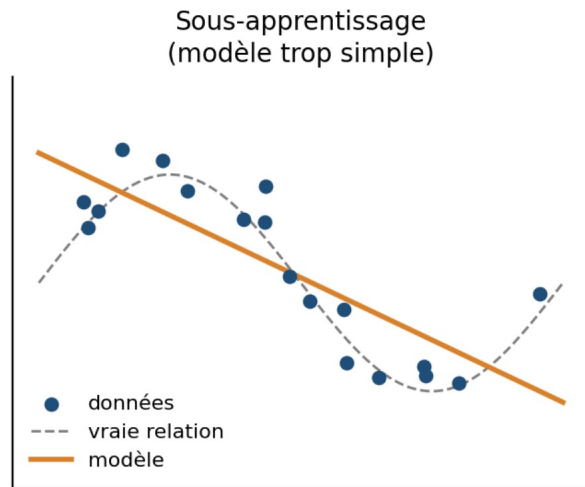
2 · GÉNÉRALISER

Un modèle qui ne se trompe jamais sur ses données d'entraînement est-il un bon modèle ?

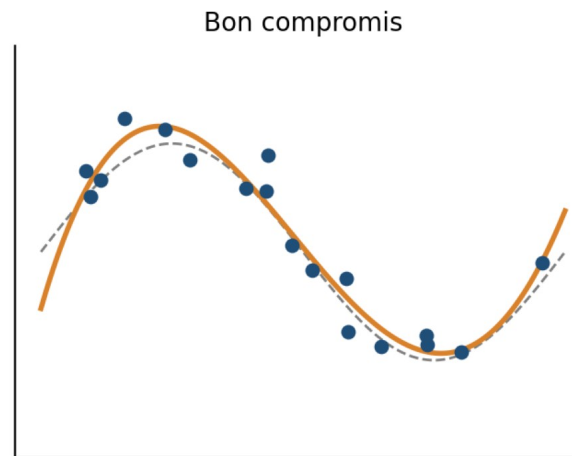
Oui, non, ça dépend ?

2 · GÉNÉRALISER

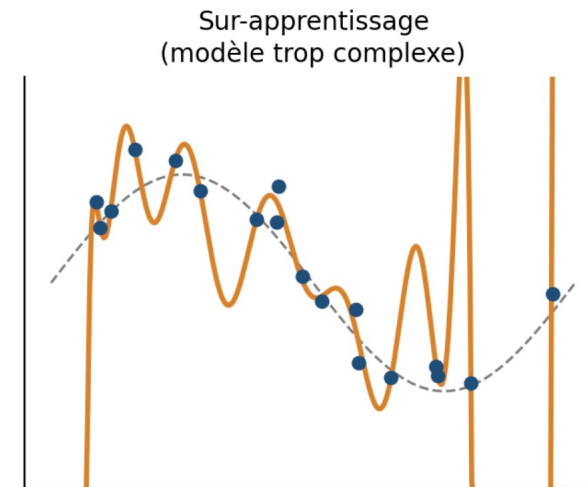
Trop simple, trop complexe



Une droite, **trop rigide** : elle rate la courbe, même sur les points qu'elle a vus. C'est le **sous-apprentissage**.

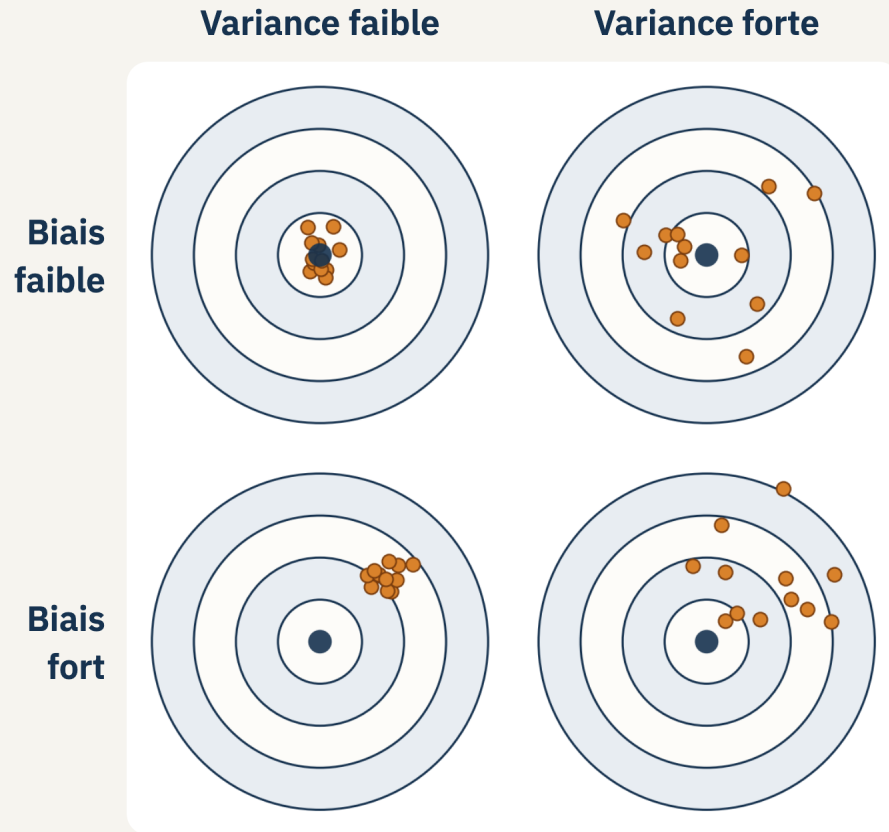


Assez souple pour suivre la tendance, sans coller au bruit : c'est le **bon compromis**.



Le modèle **récite** les données, bruit compris : parfait sur ces points, absurde ailleurs. C'est le **sur-apprentissage**.

Biais et variance : l'image de la cible



Le **centre** est la vraie valeur. Chaque **tir** est le même modèle, entraîné sur un autre échantillon.

Biais = où vise le groupe

Écart entre le milieu des tirs et le centre.

Variance = dispersion des tirs

À quel point le modèle change d'un échantillon à l'autre.

Le compromis biais-variance

$$\text{Erreur} = \text{Biais}^2 + \text{Variance} + \sigma_{\varepsilon}^2$$

Biais

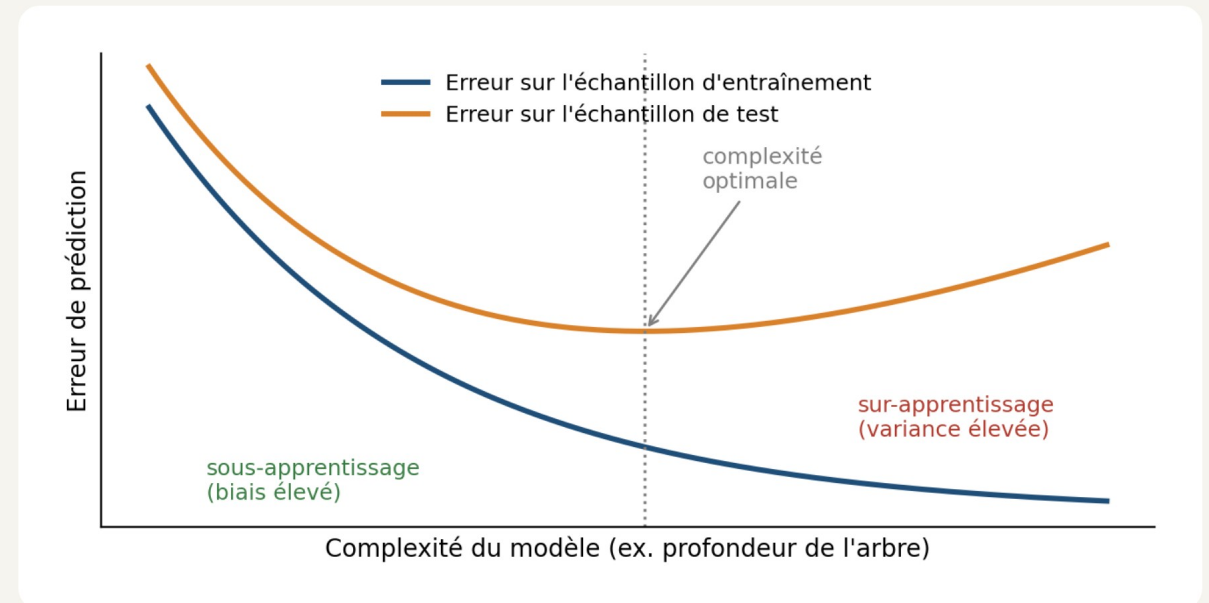
Modèle trop rigide : il rate la structure.
Sous-apprentissage.

Variance

Modèle trop instable : il change avec chaque échantillon. Sur-apprentissage.

Bruit

Incompressible, quel que soit le modèle.



2 · GÉNÉRALISER

Train et test : apprendre d'un côté, juger de l'autre

Entraînement · 80 % · 16 346 quartiers

le modèle apprend ici

Test · 20 % · 4 087

① Entraîner

Le modèle voit X **et** y pour 80% des données, et cherche la relation entre les deux.

Comme le TD et l'examen

On s'entraîne sur des exercices corrigés ; on est noté sur des sujets **inédits**. Noter un modèle sur son train, c'est noter un étudiant sur des exercices dont il a le corrigé.

② Évaluer sur le test

On cache y, le modèle prédit à partir de X seul, puis on **compare à la vraie valeur** mise de côté.

Quartier	Prédit	Réel	Erreur
n° 14416	216 600 \$	245 800 \$	29 200 \$
n° 16383	152 700 \$	137 900 \$	14 800 \$
n° 1410	117 800 \$	220 800 \$	103 000 \$

Le score résume ces erreurs, par exemple leur moyenne (partie 4).

Avant les arbres : deux repères pour comparer

① Le modèle naïf

Il prédit **la même valeur pour tous** les quartiers : la moyenne. Il ne regarde aucune variable : c'est le minimum à battre.

② La régression linéaire

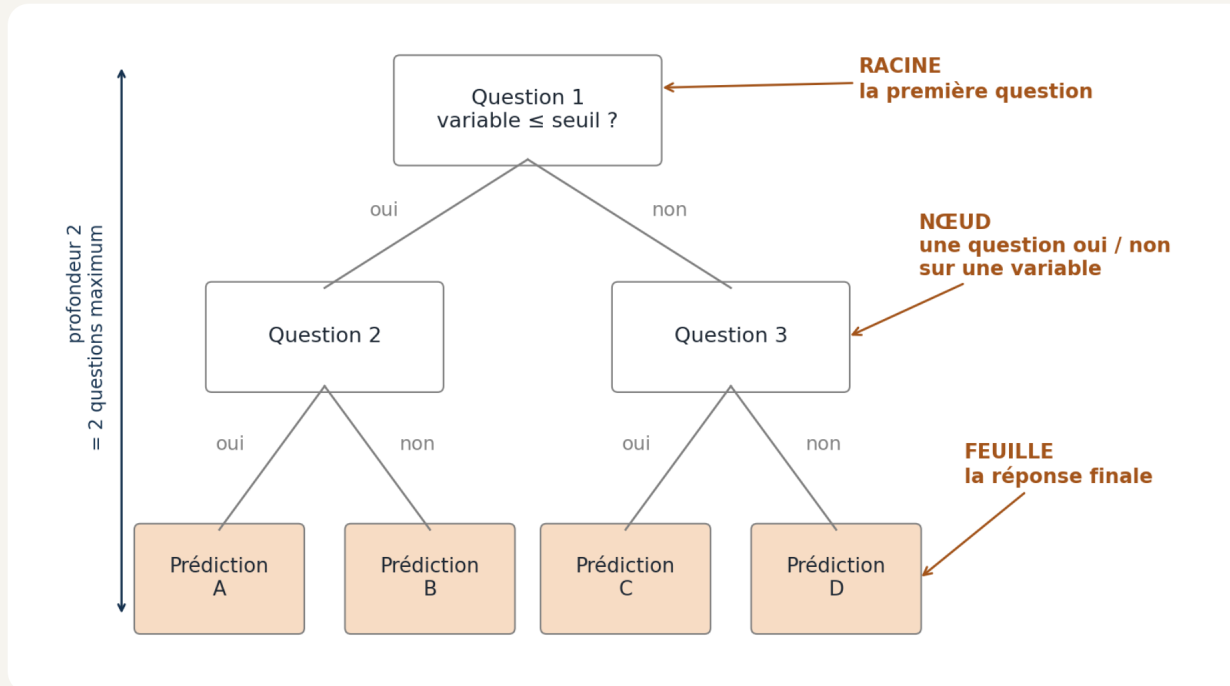
Une pente par variable (slide 7). Simple, rapide, interprétable : la **référence** classique.

La limite de la régression linéaire

Une seule pente par variable : ni effet de seuil, ni interaction (« la côte compte surtout pour les revenus modestes »), ni carte des prix. Un modèle plus flexible ne vaut la peine que s'il fait **mieux sur le test**.

3 · LES MODÈLES

L'arbre de décision : le principe



1 Prédire

On part de la racine et on répond aux questions, branche par branche, jusqu'à une feuille.

2 Construire

L'algorithme choisit lui-même les questions et les seuils, à partir des données (slide 17).

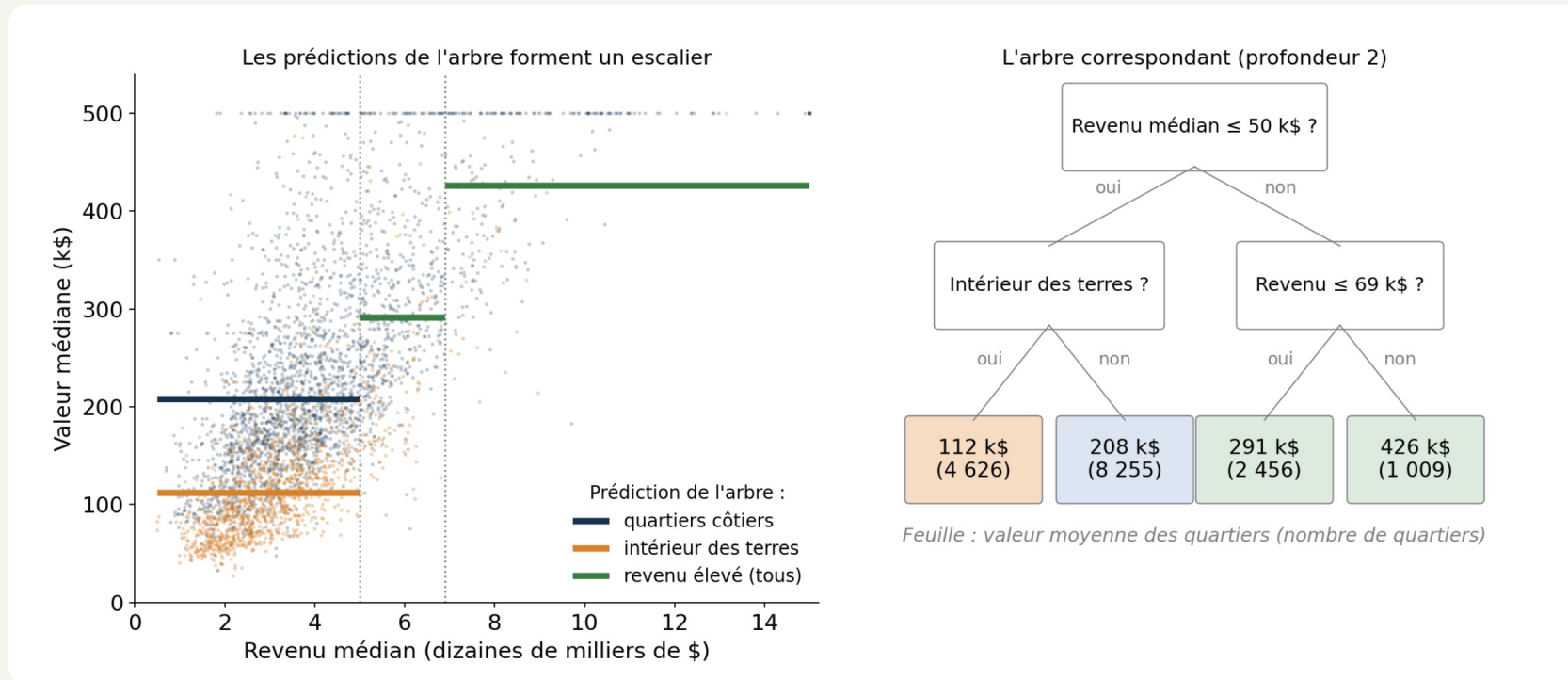
3 La réponse

En régression, une feuille prédit la **moyenne** des observations qui y tombent.

Chaque question coupe les possibilités en deux, jusqu'à la réponse.

3 · LES MODÈLES

L'arbre de décision : une suite de questions



Le « nouveau » quartier de la slide 5 (revenu 29 k\$, près de la baie) : revenu ≤ 50 k\$, pas à l'intérieur $\rightarrow$ **208 k\$**. Sa vraie valeur : 220 800 \$.

Comment l'arbre choisit ses questions

À chaque nœud, il essaie **toutes les variables et tous les seuils**, et garde la coupure qui sépare les observations en deux groupes **G** et **D** les plus homogènes, c'est-à-dire qui minimise :

$$\sum_{i \in G} (y_i - \bar{y}_G)^2 + \sum_{i \in D} (y_i - \bar{y}_D)^2$$

La prédiction d'une feuille est la **moyenne** de ses observations. Algorithme **glouton** : la meilleure question à chaque étape, sans retour en arrière.

Atouts

Lisible. Capte seuils, interactions et géographie. Pas besoin de standardiser.

Limite majeure

Très instable : changez quelques observations, l'arbre change. Sans limite, il sur-apprend.

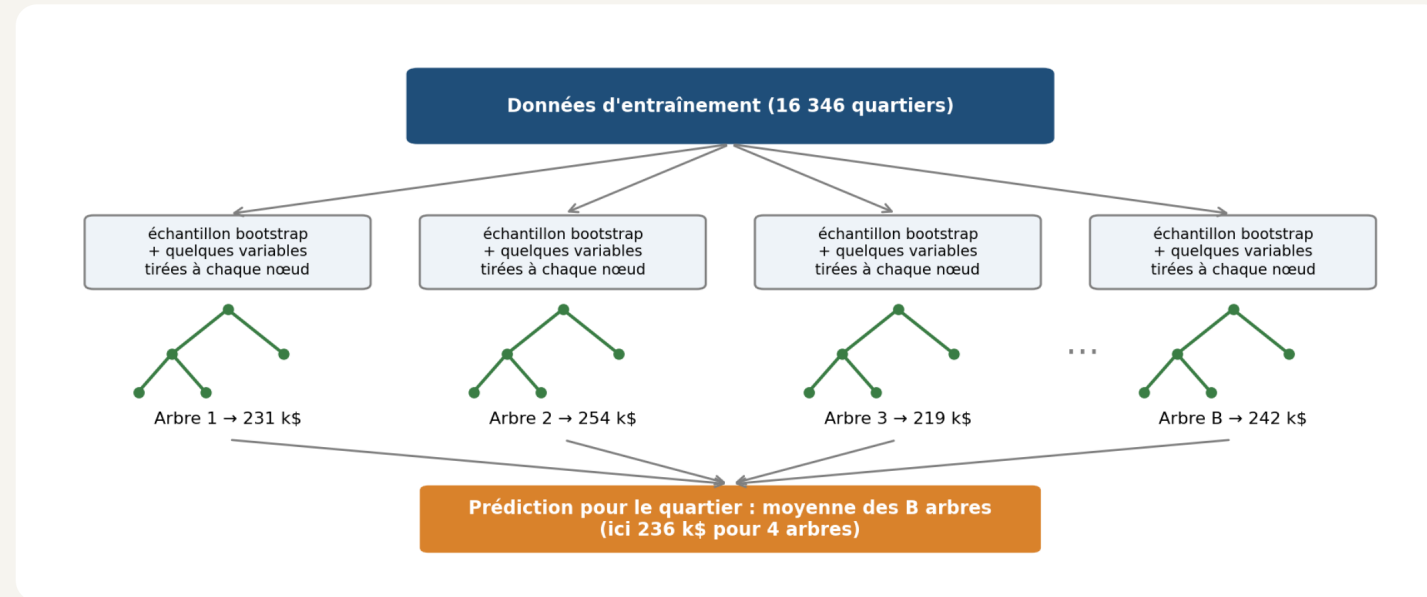
LA RANDOM FOREST · BREIMAN (2001)

Un arbre se trompe. Des centaines
d'arbres **différents** se trompent
rarement ensemble.

C'est la sagesse des foules : la moyenne de nombreux avis imparfaits mais indépendants bat souvent le meilleur avis isolé.

3 • LES MODÈLES

Random forest : deux sources de hasard



① Bagging

Chaque arbre apprend sur un échantillon **bootstrap** : tiré avec remise, il contient environ 63 % des observations.

② Tirage des variables

À chaque nœud, seule **une partie des variables**, tirée au hasard, est candidate (dans le TP : la moitié).

Prédiction pour un quartier : la **moyenne** des prédictions de tous les arbres.

3 • LES MODÈLES

Random forest : pourquoi des arbres différents ?

Cinq arbres estiment le nouveau quartier de la slide 5, dont la vraie valeur est **221 k\$**.

Des arbres qui se ressemblent

180 k\$ 182 k\$ 179 k\$ 183 k\$ 181 k\$

➡ moyenne : **181 k\$**

Ils font tous **la même erreur** : la moyenne ne corrige rien.

Des arbres différents

180 k\$ 265 k\$ 205 k\$ 240 k\$ 195 k\$

➡ moyenne : **217 k\$**

Ils se trompent **dans des sens différents** : les erreurs se compensent.

D'où le hasard injecté dans la forêt (slide 19) : chaque arbre voit d'autres quartiers et d'autres variables. Et ajouter des arbres **n'aggrave pas** le sur-apprentissage : la moyenne se stabilise.

4 • ÉVALUER

Mesurer la performance : le R^2 , calculé sur le test

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

La part des erreurs du modèle naïf (prédire la moyenne pour tous) que le modèle fait disparaître.

0 = pas mieux que la moyenne, 1 = parfait.

On le calcule sur le test.

Modèle (TP)	R ² train	R ² test
Naïf : la moyenne	0,00	0,00
Régression linéaire	0,65	0,65
Arbre sans limite	1,00	0,65
Random forest	0,96	0,82

MAE

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

L'erreur moyenne, **en dollars** : la forêt se trompe en moyenne de **32 400 \$** par quartier.

RMSE

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Aussi en dollars, mais les **grosses erreurs** pèsent plus lourd : **49 600 \$** pour la forêt.

Améliorer le R^2 : jouer sur les réglages

Réglage	Ce qu'il change	Si on l'augmente...
n_estimators	Nombre d'arbres	prédiction plus stable ; le gain plafonne, calcul plus long
max_features	Variables testées par question	arbres meilleurs un à un, mais plus semblables
max_depth	Profondeur des arbres	plus de détails (moins de biais), mais plus instable
min_samples_leaf	Taille minimale des feuilles	plus lisse (moins de variance), mais moins de détails

Ce qui compte le plus

Le **choix du modèle** pèse souvent bien plus que les réglages.
Les valeurs par défaut de scikit-learn sont un bon point de départ.

Attention

Régler en regardant le test, c'est déjà apprendre sur lui. En pratique, on règle sur une partie du train mise de côté.

TP · 55 MINUTES

À vous : prédire la valeur des logements

Données

housing.csv, depuis git : « California Housing Prices ». À placer à côté du notebook.

Notebook

TP_regression_logements.ipynb : complétez les ___ et exécutez les cellules dans l'ordre.

L'essentiel

Les **questions « Analyse »** : interprétez chaque résultat en une ou deux phrases.

SYNTHÈSE

Cinq idées à retenir

- 1 L'apprentissage supervisé apprend la relation entre X et une cible y connue, pour **prédire** y sur des cas nouveaux.
- 2 On juge un modèle sur le **test**, jamais sur les données qui ont servi à l'entraîner.
- 3 Trop simple : **biais**, sous-apprentissage. Trop complexe : **variance**, sur-apprentissage.
- 4 Un arbre est lisible mais instable ; une **random forest** moyenne des centaines d'arbres différents et réduit la variance.
- 5 Mesurer avec le **R² sur le test** (MAE et RMSE pour parler en dollars).

ANNEXE · POUR ALLER PLUS LOIN

Et quand la cible est une catégorie ? La classification

Le même cadre, avec y = défaut ou non : l'exemple du crédit, la régression logistique, l'arbre de classification, et les mesures propres à la classification (accuracy, matrice de confusion, AUC).

Une banque doit décider : prêter ou refuser ?



Ce qu'on sait du demandeur

Revenu, taux d'endettement, type de contrat, épargne, incidents de paiement passés...



Ce qu'on veut savoir

Va-t-il rembourser ? Ou, mieux : quelle est sa **probabilité de défaut** ?



Ce dont on dispose

Un historique de 5 000 prêts passés **dont on connaît l'issue** : remboursé ou défaut.

Le jeu de données : une ligne par prêt passé

X : les variables explicatives

y : la cible

Client	Revenu	Endettement	Contrat	Épargne	Incidents	Défaut
A	2 400 €	28 %	CDI	6 000 €	0	Non
B	1 500 €	41 %	CDD	800 €	2	Oui
C	3 100 €	37 %	CDI	12 000 €	0	Non
D	1 900 €	45 %	Indépendant	1 500 €	1	Oui
...	...	...	...	...	...	...
Nouveau	2 000 €	39 %	CDD	1 200 €	1	?

Pour les 5 000 prêts de l'historique, on connaît **X et y** : c'est là-dessus qu'on apprend. Pour le nouveau client, on ne connaît que **X** : le modèle doit deviner le **?**

Et si y vaut 0 ou 1 ? La régression logistique

$$P(\text{défaut} | x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p)}}$$

Une courbe en S

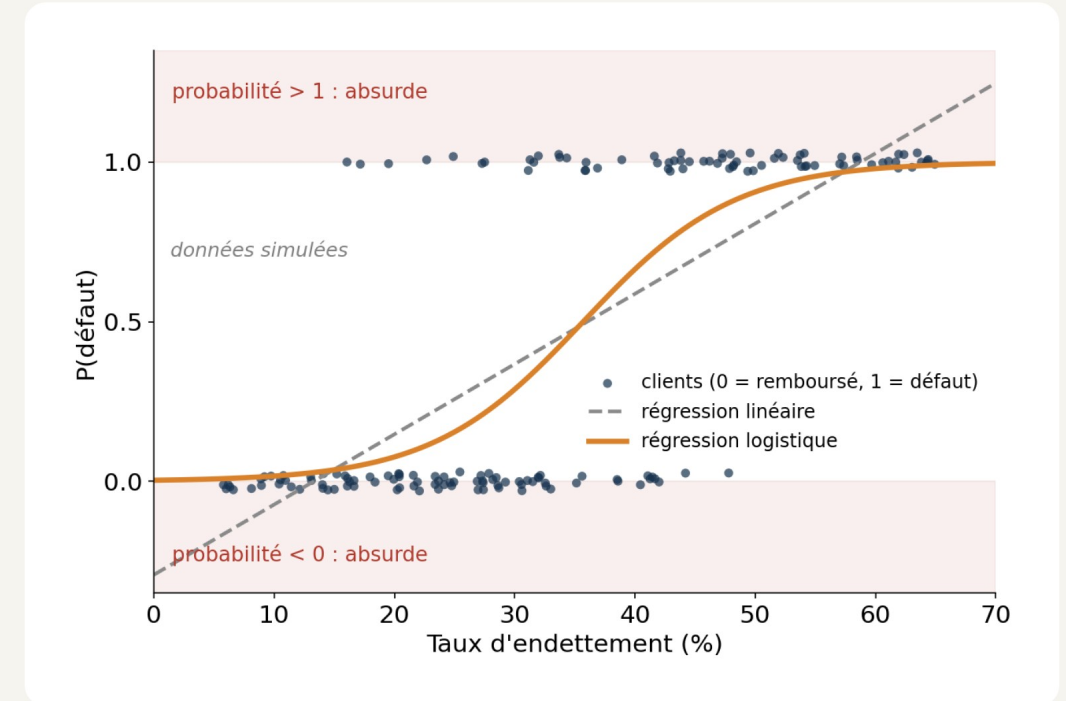
La même combinaison linéaire qu'en régression, transformée en probabilité toujours comprise entre 0 et 1.

Estimée par maximum de vraisemblance

et non par les MCO. Chaque $\exp(\beta)$ est un odds ratio : il multiplie la cote du défaut.

De la probabilité à la décision

On prédit « défaut » si la probabilité dépasse un seuil.



La régression logistique, référence à battre

$$P(\text{défaut} | x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p)}}$$

Atouts

Simple et rapide. Coefficients interprétables. Bien acceptée par les régulateurs : le modèle historique du scoring.

Limite

Le logit est **linéaire** en x : pas d'effet de seuil (le risque qui s'envole au-delà de 35 % d'endettement), pas d'interaction, sauf à les construire à la main.

Comment l'arbre choisit ses questions

À chaque nœud, il essaie **toutes les variables et tous les seuils**, et garde la coupure qui rend les deux groupes les plus **homogènes**.

$$G = 1 - \sum_k p_k^2$$

$G = 0$: nœud pur
 $G = 0,5$: moitié-moitié

Algorithme **glouton** : la meilleure question à chaque étape, sans retour en arrière, jusqu'à une profondeur maximale.

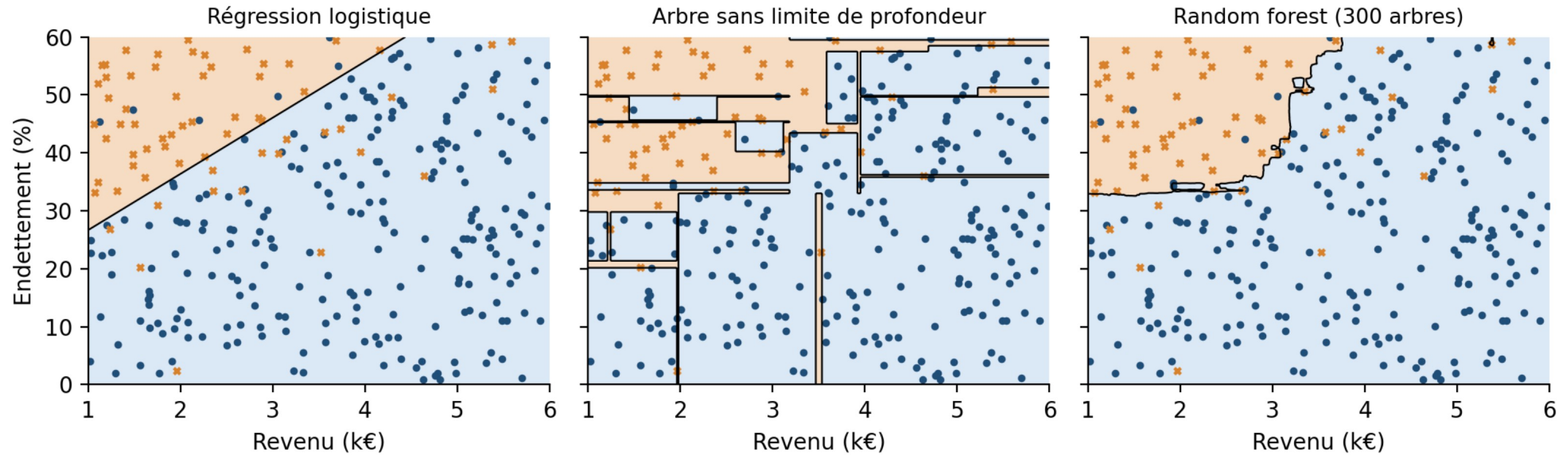
Atouts

Lisible par un comité de crédit. Capte seuils et interactions. Pas besoin de standardiser.

Limite majeure

Très instable : changez quelques données, l'arbre change entièrement. Profond, il sur-apprend.

Trois modèles, les mêmes données



Une droite : rate l'effet de seuil.

Découpe le bruit : sur-apprentissage.

Retrouve la zone à risque.

La matrice de confusion et ce qu'on en tire

Réalité ↓ Prédit →	Prédit : remboursé	Prédit : défaut
Réalité : remboursé	798 vrais négatifs bon client accepté	42 faux positifs bon client refusé
Réalité : défaut	87 faux négatifs défaut non détecté	73 vrais positifs défaut détecté

TP de classification : random forest, seuil 0,5, 1 000 clients de test (160 défauts réels)

87 %

Accuracy = (VP + VN) / total

Part de décisions justes. Trompeuse : prédire « remboursé » pour tous ferait déjà 84 %.

63 %

Précision = VP / (VP + FP)

Parmi les clients signalés à risque, combien font vraiment défaut ? 73 sur 115.

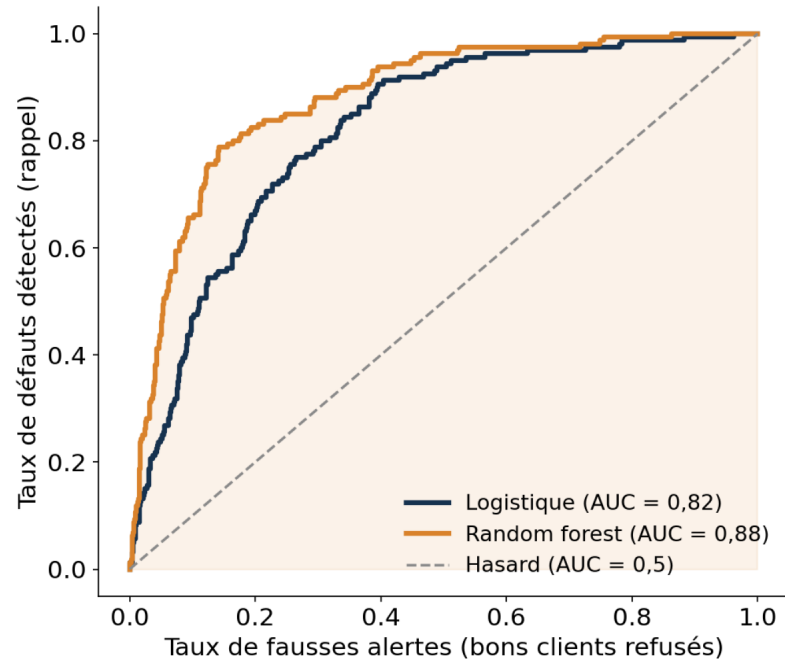
46 %

Rappel = VP / (VP + FN)

Parmi les vrais défauts, combien ont été détectés ? 73 sur 160 : moins de la moitié.

Tout dépend du **seuil** (ici 0,5) : l'abaisser détecte plus de défauts (rappel ↑) mais refuse plus de bons clients (précision ↓).

L'AUC : juger le classement des clients



AUC : savoir classer les clients

La courbe ROC montre tous les seuils à la fois. L'AUC est l'aire sous la courbe : **0,5 = hasard**, **1 = classement parfait**. Dans le TP : logistique 0,82, forêt 0,88.

Lecture intuitive

L'AUC est la probabilité qu'un client défaillant tiré au hasard ait un score de risque plus élevé qu'un bon payeur. En scoring, on parle aussi de **Gini = 2 × AUC – 1**.